

Semana 4 — GameManager e SaveManager (Autoload/Singleton)

Semana 4

GameManager e SaveManager (Autoload/Singleton)

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade II — Construir Sistemas (Semanas 4–7)

Projeto: Vertical Slice *O Templo Esquecido*

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender Autoload/Singleton como mecanismo nativo do Godot para estado global compartilhado entre cenas
- Construir um `GameManager` que centraliza regras de partida e estado compartilhado, e dar a ele sua primeira responsabilidade concreta: `spawn_player()` (posicionar o Player no `PlayerStart` do nível)
- Construir um `SaveManager` que centraliza persistência de dados entre cenas

Semana 4 — GameManager e SaveManager (Autoload/Singleton)

ENCONTRO 1

Autoload/Singleton e o GameManager

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão do Encontro 2 da Semana 3 (build exportado, encerramento do Módulo 1) (15 min)
- Introdução: onde vive o estado que não pertence a nenhuma cena específica (20 min)
- Demonstração: criação da Orchestration `GameManager`, registro como Autoload e `spawn_player()` com o `PlayerStart` (35 min)
- Laboratório: cada estudante cria o `GameManager` (Orchestration), o `PlayerStart` no nível e `spawn_player()` (40 min)
- Desafio: adicionar uma variável de estado de partida própria (15 min)
- Feedback e fechamento (5 min)

Semana 4 — GameManager e SaveManager (Autoload/Singleton)



Onde vive um dado que várias cenas precisam compartilhar, mas que não pertence a nenhuma delas?

Pense em pontuação, condição de vitória ou referência ao jogador ativo.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

O Problema Universal do Estado Global

Toda engine que trabalha com múltiplas cenas precisa resolver o mesmo problema.

- Um dado guardado dentro de uma Scene comum desaparece quando ela é trocada
- Pontuação, condição de vitória e referência ao jogador ativo não pertencem a nenhuma cena específica
- Toda engine moderna oferece um mecanismo dedicado para esse tipo de estado

Autoload/Singleton no Godot

- **Autoload** — script (`.gd`), Orchestration (`.torch`) **ou** cena (`.tscn`) registrado em Project Settings, instanciado automaticamente na raiz da árvore de cenas
- Acessível globalmente por nome, de qualquer script ou Orchestration do projeto
- Sobrevive a qualquer troca de cena
- O Godot formaliza como recurso de primeira classe do editor

GameManager — Papel no Projeto

- Reúne regras de partida e estado compartilhado em um único ponto
- Construído como Orchestration com classe base `Node`, registrada diretamente como Autoload
- GDScript aparece apenas como **implementação de referência** da lógica do grafo
- Local no projeto: `orchestrations/autoload/game_manager.torch`

Estado Global no Godot × Unity

Godot

- Autoload/Singleton, registrado em Project Settings (script `.gd` ou Orchestration `.torch`)
- Instância única garantida pelo editor
- `GameManager` reúne `GameMode` + `GameState`

Unity

- Sem mecanismo formal equivalente
- Resolvido por convenção: `DontDestroyOnLoad` ou singleton manual em C#
- Depende da disciplina da equipe

PlayerStart — o GameManager Decide Onde o Player Nasce

- Toda engine precisa decidir a posição inicial do jogador ao carregar um nível
- Unreal formaliza: actor **PlayerStart** (a coordenada) + **GameMode.ChoosePlayerStart** (a decisão)
- Godot não formaliza: um **Marker3D** no nível guarda a coordenada; o **GameManager** a consulta
- **GameManager.spawn_player()** (função pública do grafo) lê o grupo **player_start** e reposiciona o Player — **nenhuma coordenada vive no GameManager**

Demonstração — Criando o GameManager

O que será construído:

- Orchestration `orchestrations/autoload/game_manager.torch`, classe base `Node`
- Registro direto em **Project Settings > Autoload**
- Teste de acesso a partir do Player, com um nó Print / `print(GameManager)`

Por quê: primeiro Autoload do projeto, base de todo o Módulo 2.

Imagem sugerida

*Objetivo: mostrar a aba Autoload de Project Settings com o **GameManager** recém-registrado.*

Enquadramento: captura de tela da janela Project Settings, aba Autoload.

Elementos presentes: campo Path apontando para

***res://orchestrations/autoload/game_manager.torch**, campo Node Name como **GameManager**, caixa Enable marcada.*

*Destaque visual: a linha do **GameManager** na lista de Autoloads.*

Legenda sugerida: "GameManager registrado e habilitado como Autoload em Project Settings."

Laboratório — GameManager

Cada estudante replica, no próprio projeto:

1. Pasta `orchestrations/autoload/` criada, conforme `PROJECT_ARCHITECTURE.md`
2. Orchestration `game_manager.torch`, classe base `Node`
3. Registro direto em **Project Settings > Autoload**
4. Teste de acesso ao `GameManager` a partir do Player
5. `Marker3D` `PlayerStart` no nível (grupo `player_start`); Player no grupo `player`
6. Função pública `spawn_player()` no `GameManager` + chamada no `_ready()` do nível; mover o marcador muda o spawn
7. Remoção do teste (nó Print / `print()`) ao final

Boas Práticas — Autoload

- Manter toda Orchestration de Autoload dentro de `orchestrations/autoload/`, nunca solta na raiz de `res://`
- Registrar, desde o primeiro grafo, uma nota curta explicando sua responsabilidade
- Testar o acesso ao Autoload a partir de uma cena diferente da que está sendo editada
- Nunca duplicar em uma Scene um dado que já pertence ao `GameManager`



Desafio — Encontro 1

Adicione ao `GameManager` uma variável de estado de partida própria, não demonstrada em aula — um contador de tentativas, um estado de progresso ou uma flag de evento.

OBJETIVOS

Justifique em uma nota no grafo por que essa variável pertence ao `GameManager` e não a uma Scene específica.

Fechamento — Encontro 1

- `GameManager` (Orchestration) criado e registrado como Autoload, com acesso validado a partir do Player
- `PlayerStart` (Marker3D) no nível; `spawn_player()` posiciona o Player nele ao carregar — sem coordenada no grafo
- Variável de estado própria do desafio adicionada, com justificativa anotada no grafo
- Nível e build da Semana 3 intactos (apenas grupos + chamada de spawn no `_ready()`)
- Próximo passo: input centralizado no Player e SaveManager, no Encontro 2

Semana 4 — GameManager e SaveManager (Autoload/Singleton)

ENCONTRO 2

Input no Player e o SaveManager

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (GameManager registrado como Autoload) (10 min)
- Introdução: input centralizado no Player e o problema da persistência (20 min)
- Demonstração: criação da Orchestration SaveManager e variável persistente (35 min)
- Laboratório: cada estudante cria seu SaveManager (Orchestration) e testa persistência (45 min)
- Desafio: dado próprio persistindo entre cenas (20 min)
- Feedback e fechamento (5 min)

Semana 4 — GameManager e SaveManager (Autoload/Singleton)



Por que o Godot não separa "quem controla" de "o que é controlado", como a Unreal faz?

Pense em como o Player já lê input desde a Semana 2.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Input de Alto Nível no Player

- Unreal separa formalmente PlayerController (lê input) de Pawn/Character (é controlado)
- Godot concentra a leitura de input de alto nível no próprio Node do Player
- Não é uma limitação — é uma escolha arquitetural válida
- Reduz camadas de indireção para ações simples de gameplay

Input Centralizado no Godot × Unity

Godot

- Sem separação nativa Pawn/Controller
- Input concentrado no próprio Player

Unity

- Também sem separação formal
- Input tipicamente no script do personagem ou Input System conectado a ele

O Problema Universal da Persistência

Toda engine multi-cena precisa de um lugar para guardar dados que sobrevivem à troca de cena — antes mesmo de qualquer gravação em disco.

- **GameManager** guarda regras e estado da partida atual
- **SaveManager** guarda dados que precisam sobreviver à troca de cena
- Separação evita que um único Autoload acumule responsabilidades demais

SaveManager — Segundo Autoload do Projeto

- Orchestration independente, classe base `Node`
- Registrada separadamente em **Project Settings > Autoload**, direto pelo `.torch`
- Local no projeto: `orchestrations/autoload/save_manager.torch`
- Guarda dados que sobrevivem à troca de cena, em memória

Persistência entre Cenas no Godot × Unity

Godot

- Segundo Autoload dedicado (`SaveManager`), Orchestration `.torch`
- Sobrevivência garantida pelo registro em Project Settings

Unity

- Singleton manual, muitas vezes o mesmo objeto do "game manager"
- Sobrevivência via `DontDestroyOnLoad`

Demonstração — SaveManager e Persistência

O que será construído:

- Orchestration `save_manager.torch`, com uma variável persistente (ex.: `itens_coletados`) e função pública para alterá-la
- Scene de teste `level_teste_persistencia.tscn`
- Troca de cena real, validando que o valor não é perdido

Por quê: um Autoload só demonstra sua utilidade quando testado sob o cenário que ele resolve.

Imagem sugerida

*Objetivo: mostrar a aba Autoload de Project Settings com dois registros — **GameManager** e **SaveManager** — lado a lado.*

Enquadramento: captura de tela da janela Project Settings, aba Autoload.

*Elementos presentes: lista com os dois Autoloads, coluna Path (ambas apontando para arquivos **.torch**) e coluna Node Name.*

Destaque visual: as duas linhas, reforçando que são Autoloads independentes.

Legenda sugerida: "GameManager e SaveManager registrados como Autoloads independentes ao final da Semana 4."

Laboratório — SaveManager

Cada estudante replica, no próprio projeto:

1. Orchestration `save_manager.torch`, classe base `Node`
2. Registro direto em **Project Settings > Autoload**
3. Variável persistente e função pública para alterá-la
4. Scene de teste com troca real de cena, validando a persistência
5. Reversão da Main Scene para `level_exploration.tscn`

Boas Práticas — SaveManager

- Manter `GameManager` e `SaveManager` como Orchestrations totalmente independentes
- Registrar, na nota de topo de cada Autoload, qual tipo de dado ele guarda
- Sempre testar persistência com uma troca de cena real, nunca apenas revisando o grafo
- Nomear variáveis de forma que já sugiram o que representam no Vertical Slice final



Desafio — Encontro 2

Defina e implemente, no `SaveManager`, um dado próprio que deve persistir entre cenas — pontuação, item coletado ou estado de progresso, diferente do demonstrado.

OBJETIVOS

Valide a persistência com uma troca de cena real dentro do projeto.

Checkpoint — Base do Módulo 2

Ao final da semana, cada estudante possui:

- **GameManager** (Autoload), com variável de estado de partida própria
- **SaveManager** (Autoload), independente, com dado persistindo entre cenas
- Nível, Player e build da Semana 3, sem nenhuma alteração

Fechamento — Encontro 2

- Discussão sobre input centralizado no Player concluída
- `SaveManager` criado, registrado e testado com troca real de cena
- Desafio do grupo (dado próprio persistente) implementado
- Módulo 2 avança com dois Autoloads funcionais

Resultado Esperado da Semana

- `GameManager` (Autoload), com regras e estado de partida compartilhado e `spawn_player()` (o "ChoosePlayerStart" do projeto)
- `SaveManager` (Autoload), independente, com dado de progresso persistindo entre cenas
- Turma relaciona Autoload/Singleton à ausência de equivalente formal na Unity
- Turma compreende input centralizado no Player como escolha arquitetural válida

Checklist da Semana

- [] Orchestration `game_manager.torch` criada (classe base `Node`), registrada diretamente como Autoload
- [] `PlayerStart` (`Marker3D`, grupo `player_start`) no nível; função pública `spawn_player()` posiciona o Player e é chamada no `_ready()` do nível
- [] Variável de estado própria do desafio adicionada ao `GameManager`
- [] Orchestration `save_manager.torch` criada (classe base `Node`), registrada diretamente como Autoload
- [] Variável persistente implementada e validada com troca real de cena
- [] Main Scene revertida para `level_exploration.tscn` após o teste
- [] Dado próprio do desafio do `SaveManager` implementado e validado

Próximos Passos — Semana 5

Os dois Autoloads construídos nesta semana são a base direta da Semana 5:

- Contrato `Interactable` e Signals para comunicação desacoplada entre sistemas
- Primeiro objeto interativo do Vertical Slice (porta ou equivalente)

Leitura recomendada: Godot Docs — Singletons (Autoload), GDScript; Orchestrator Docs — Autoloads; Unity Manual (consulta comparativa) — DontDestroyOnLoad.